

EASNFW Requirements

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Product Overview	2
1.3.1	Product Perspective	2
1.3.2	Product Functions	3
1.3.3	System Architecture	3
1.3.4	User Characteristics	3
1.3.5	Limitations	4
1.4	Definitions	4
1.5	Firmware Images	4
1.6	Payloads	5
2	Requirements	5
2.1	Initialization	5
2.1.1	REQ-001	5
2.1.2	REQ-002	5
2.1.3	REQ-003	5
2.1.4	REQ-004	5
2.1.5	REQ-005	6
2.1.6	REQ-006	6
2.1.7	REQ-007	6
2.1.8	REQ-008	6
2.1.9	REQ-009	6
2.1.10	REQ-010	6
2.1.11	REQ-011	6
2.1.12	REQ-012	6
2.1.13	REQ-013	7

1 Introduction

1.1 Purpose

The purpose of the software whose requirements are specified in this document is to control the hardware of an ecoacoustic sensing node.

1.2 Scope

This document specifies the requirements for the software product designated as EASNFW, from **ecoacoustic sensing node firmware**.

Within this document, EASNFW may also be referred to as *the firmware*. The ecoacoustic sensing node as a whole, comprising all software and hardware components of the device, is designated as EASN and may also be referred to as *the system* or *the sensing node*.

The purpose of EASNFW is to control the hardware of EASN in order to:

- acquire ecoacoustic data through the system sensors;
- process the acquired data; and
- transmit the acquired and processed data to a cloud platform.

The cloud platform may be referred to as *the cloud platform* or simply *the cloud*. The acquired data is intended for use in ecoacoustic studies.

1.3 Product Overview

1.3.1 Product Perspective

The system is intended to integrate a distributed ecoacoustic data acquisition infrastructure. This infrastructure is composed of a set of sensing nodes and a cloud platform.

The cloud platform stores the data acquired by the sensing nodes and provides support for visualizing and analyzing the collected data.

The system hardware is based on the boards nRF5340 Audio DK and nRF9151 SMA DK from Nordic Semiconductor.

EASNFW will run on and control the nRF5340 System-on-Chip (SoC) from the nRF5340 Audio DK and the nRF9151 System-in-Package (SiP) from the nRF9151 SMA DK. EASNFW is hence subdivided into two different components: one that will run on nRF5340 and be mainly responsible for interfacing with the system sensors, and processing, storing and retrieving the data acquired through them, and is therefore designated as EASNFW-SENSOR; and another one that will run on nRF9151 and be mainly responsible for interfacing with the cloud, and is therefore designated as EASNFW-CLOUD. EASNFW-SENSOR and EASNFW-CLOUD will communicate over SPI, with EASNFW-SENSOR as the controller and EASNFW-CLOUD as the peripheral.

EASNFW-CLOUD will communicate with the cloud using HTTP over LTE-M through the built-in LTE-M modem of nRF9151. Communication is unidirectional: data flows from the sensing node to the cloud, but not from the cloud to the sensing node.

User interfaces EASNFW-SENSOR will provide a command line interface (CLI) for verifying specific features. Such interface will only be enabled when working with the “CLI verification image” (see Section 1.5).

Both EASNFW-SENSOR and EASNFW-CLOUD will log their own activities through the serial-USB interface of their respective boards when logging is enabled.

1.3.2 Product Functions

The main functions that will be performed by EASNFW are described in Table 2, alongside the firmware component responsible for each function.

Function	Responsible firmware component
Acquire audio data	EASNFW-SENSOR
Process audio data	EASNFW-SENSOR
Acquire environmental data	EASNFW-SENSOR
Transmit acquired environmental data and processed audio data to the cloud	EASNFW-CLOUD

Table 1: EASNFW functions description

1.3.3 System Architecture

The diagram in Figure 1 provides an overview of the system architecture, showing the main blocks that compose and communicate with it, and how they interface with EASNFW.

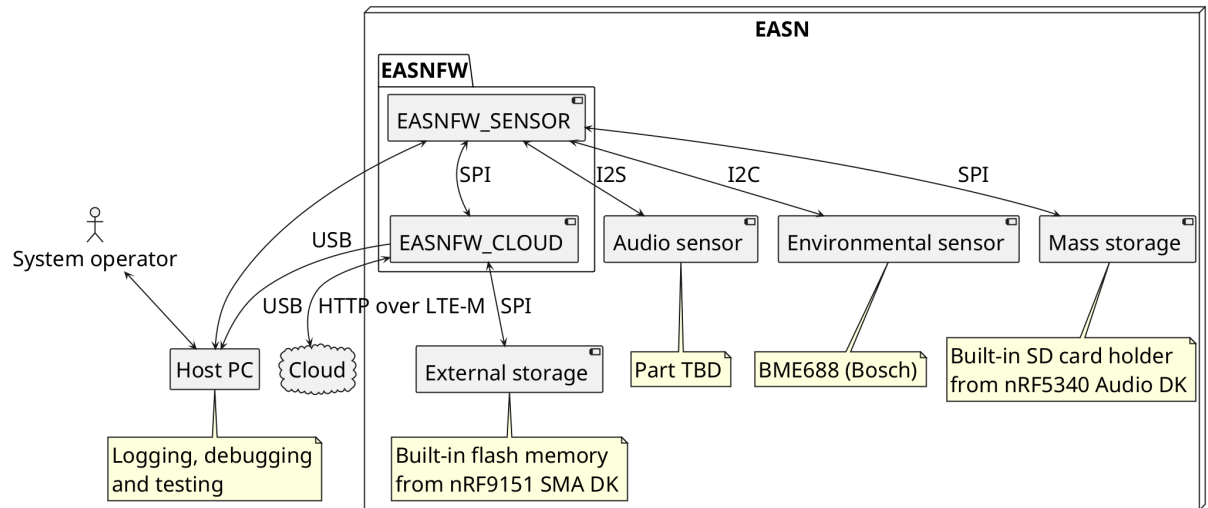


Figure 1: System architecture overview

1.3.4 User Characteristics

Users of EASN and EASNFW are expected to be familiar with general concepts related to:

- embedded systems;
- Internet of Things (IoT);
- wireless communication; and
- digital signal processing (DSP).

1.3.5 Limitations

EASNFW functions and their implementations are constrained by:

- the computational, memory, communication, and energy resources of the target hardware; and
- the interfaces provided by the cloud platform.

1.4 Definitions

Audio track

An ordered collection of audio samples captured sequentially over a continuous period of `PARAM_TRACK_LEN` seconds, which will be configurable via `Kconfig`.

Audio processing algorithm

The algorithm used to process the recorded audio tracks.

Processed audio track

The output, or collection of outputs, produced by the audio processing algorithm for a given audio track.

Environmental sample set

A set of environmental variables sampled within a short interval. The set consists of temperature, humidity, pressure and volatile organic compounds (VOCs).

Self-test sequence

Sequence of tests executed by EASNFW to verify the correct working of individual software modules, as well as the integration between them. It is broken down into the independent self-test sequence, which is a firmware component-specific sequence, meaning that each of EASNFW-SENSOR and EASNFW-CLOUD has its own independent self-test sequence; and the shared self-test sequence, which is carried out by EASNFW-SENSOR and EASNFW-CLOUD together.

Verification images

Set of images intended to be used solely for verification, comprising the regression verification image and the CLI verification image (see Section 1.5).

1.5 Firmware Images

Production image

Image of the firmware to be used for deployment. Contains all the business logic and logging is disabled. Built when the `BUILD_PROD` flag is set.

Debugging image

Image of the firmware to be used for debugging, it is the same as the production image but with logging enabled. Built when the `BUILD_DEBUG` flag is set.

Development image

Image of the firmware to be used for development, it is the same as the debugging image, but with the self-test sequence included. Built when the `BUILD_DEVELOP` flag is set.

Regression verification image

Image of the firmware that contains only the self-test sequence with logging enabled. Built when the `BUILD_REGR` flag is set.

CLI verification image

Image of the firmware to be used for verifying specific product functions, such as the audio acquisition and processing, through a command-line interface (CLI) running on EASNFW-SENSOR. Built when the `BUILD_CLI` flag is set. Only valid for EASNFW-SENSOR.

1.6 Payloads

Ecoacoustic data payload

A payload transmitted periodically to the cloud that consists of a processed audio track paired with an environmental sample set and a timestamp indicating when the corresponding audio track was recorded.

Power-on payload

A payload transmitted to the cloud during EASNFW initialization that consists of the respective revisions of EASNFW-SENSOR and EASNFW-CLOUD and the reason why the device was last powered off.

2 Requirements

2.1 Initialization

2.1.1 REQ-001

When the system is powered on, the firmware shall execute the initialization sequence, which comprises: (a) initializing critical peripherals¹; (b) running the self-test sequences²; (c) establishing an Internet connection; (d) transmitting the power-on payload to the cloud; and (e) transmitting to the cloud any pending ecoacoustic data payloads from previous boot cycles.

2.2 Data Acquisition

2.2.1 REQ-002

After the initialization sequence is successfully completed, the firmware shall periodically sample the audio into PARAM_TRACK_LEN-second audio tracks with a PARAM_INTER_TRACK_INTER second interval between the sampling of two consecutive audio tracks.

Parameters for audio sampling:

- Number of channels: 1
- Sample rate: PARAM_AUDIO_SAMPLE_RATE kHz
- Bandwidth: PARAM_AUDIO_SAMPLE_BW_LO Hz - PARAM_AUDIO_SAMPLE_BW_HI kHz
- Sample width: PARAM_AUDIO_SAMPLE_WIDTH bits

2.2.2 REQ-003

The firmware shall detect the loss of any audio samples and restart the audio track sampling in such an event.

2.2.3 REQ-004

Before the beginning of the sampling of each audio track, the firmware shall capture a timestamp, including the date and time with at least a 1-second resolution, and sample the environmental data.

¹(a) is already taken care of by Zephyr.

²(b) is only valid for the firmware images that include the self-test sequences.

2.3 Data Processing

2.3.1 REQ-005

The firmware shall process the audio tracks according to the audio processing algorithm.

2.4 Cloud Communication

2.4.1 REQ-006

After an audio track is processed, the firmware shall send the processed audio track, alongside the timestamp and environmental data that were captured before the beginning of the sampling of that audio track, to the cloud.

2.4.2 REQ-007

When the transmission of the power-on payload to the cloud fails, the firmware shall retry transmitting it up to `PARAM_NUM_RETRY_TX_CLOUD` times with an exponential backoff model defined by $t[s] = 2^c$, where c is the number of times that the transmission has been retried.

2.4.3 REQ-008

When the transmission of an ecoacoustic data payload to the cloud fails, the firmware shall retry transmitting it until the free space of the mass storage is insufficient to store the data relative to another audio track. The firmware shall use an exponential backoff model defined by $t[s] = 2^c$, where c is the number of times that the transmission has been retried.

2.4.4 REQ-009

The firmware shall be designed so that communication parameters can be adapted to the regulatory requirements of different geographical regions.

2.5 Data Persistence

2.5.1 REQ-010

The firmware shall persist ecoacoustic data payloads that have not been sent to the cloud across boot cycles.

2.5.2 REQ-011

The firmware shall persist details of fatal failures across boot cycles.

2.6 Power Management

2.6.1 REQ-012

When the firmware is waiting for triggers to take action, the firmware shall enter an energy-saving state.

2.7 Configuration

2.7.1 REQ-013

The following parameters shall be configurable at compile-time via Kconfig:

- `PARAM_TRACK_LEN` (seconds)

- PARAM_INTER_TRACK_INTERVAL (seconds)
- PARAM_NUM_RETRY_TX_CLOUD
- PARAM_AUDIO_SAMPLE_RATE (kHz)
- PARAM_AUDIO_SAMPLE_BW_LO (Hz)
- PARAM_AUDIO_SAMPLE_BW_HI (kHz)
- PARAM_AUDIO_SAMPLE_WIDTH (bits)

2.8 Observability

2.8.1 REQ-014

The firmware shall log information relevant to verifying correct system operation and to identifying and diagnosing failures, in a way that the logs can be monitored in real-time from a host personal computer (PC) through USB.

2.9 Verification

2.9.1 REQ-015

The CLI image of the firmware shall provide a command to sample an audio track with the specified length, passed as a parameter, and return the captured samples.

2.9.2 REQ-016

The CLI image of the firmware shall provide a command to sample an audio track with the specified length, passed as a parameter, process it according to the audio processing algorithm, and return both the captured samples and the processed audio track.

2.9.3 REQ-017

The self-test sequence shall verify that audio data can be sampled from the audio sensor.

2.9.4 REQ-018

The self-test sequence shall verify that environmental data can be sampled from the environmental sensor.

2.9.5 REQ-019

The self-test sequence shall verify that data can be written to and read from the mass storage.

2.9.6 REQ-020

The self-test sequence shall verify that data can be written to and read from the external storage.

2.9.7 REQ-021

The self-test sequence shall verify the communication between EASNFW-SENSOR and EASNFW-CLOUD.

2.9.8 REQ-022

The self-test sequence shall verify that data can be transmitted to the cloud.

2.9.9 REQ-023

The self-test sequence shall verify that the reason for the last power-off can be retrieved.

2.9.10 REQ-024

The self-test sequence shall log the results of the executed tests over USB for debugging purposes.

3 Verification

Requirements might be verified through analysis of:

- firmware logs;
- data received by the cloud platform;
- data returned by the CLI verification image; and/or
- source code.

For requirements that specify different outcomes under different conditions, each condition and its corresponding outcome will be independently tested.

The firmware product will include a report with details about the verification performed for each requirement.